Le architetture di un Computer

I dati grezzi

I dati si riferiscono a fatti grezzi, non elaborati, che vengono raccolti da diverse fonti. Questi fatti possono essere numeri, simboli, caratteri o anche osservazioni.
I dati da soli non hanno alcun significato o contesto fino a quando non vengono elaborati o interpretati.
Esempi di "dati" includono:

- un valore (ad esempio, 35),
- una serie di misurazioni,
- un nome,
- un colore,
  … e così via.

> 🎟 **Punto chiave:**
>
> **I dati sono disorganizzati e non hanno significato diretto da soli.**

Informazione e conoscenza

L'informazione è ciò che ottieni quando i dati vengono elaborati e organizzati, dando un contesto ai dati.
Esempio di "informazione":

- un valore associato ai gradi Celsius (ad esempio, 35°C) rappresenta un valore di temperatura.

La conoscenza è la comprensione e l'intuizione acquisita dall'interpretazione delle informazioni, rendendole utili per prendere decisioni. La conoscenza risponde a domande come "cosa," "chi," "quando" e "quanto."
Esempio di "conoscenza":

- un valore di temperatura come 35°C diventa conoscenza quando viene interpretato come un ambiente caldo.

La gestione dell'informazione:

il Codice Binario

Qualsiasi informazione memorizzata o recuperata da un dispositivo (come computer desktop, laptop o smartphone) deve essere rappresentata in un linguaggio che il dispositivo possa comprendere.
Questo linguaggio è noto come **Codice Binario:**
è definito come un sistema di codifica che utilizza le cifre binarie `0` e `1` per rappresentare lettere, cifre o altri caratteri in un computer o in altri dispositivi elettronici, come smartphone e tablet.

Più specificamente, l'elemento fondamentale del codice binario è il **BIT (cifra binaria)**:
è l'unità di base di informazione utilizzata dai dispositivi per gestire tutti i tipi di dati.
In altre parole:
un'unità minima di informazione che può assumere due soli stati: `0` oppure `1`.
Un singolo BIT può esistere in uno dei **2 stati**: `0` o `1`.

Bit e Combinazioni:

Quando utilizziamo più bit insieme, il numero di combinazioni cresce in maniera esponenziale.
Infatti, con **N bit** possiamo rappresentare **combinazioni distinte**.

Esempi:

o 1 bit: = 2 combinazioni (`0` , `1`)
o 2 bit: = 4 combinazioni (`00` , `01` , `10` , `11`)
o 3 bit: = 8 combinazioni (000, 001, 010, 011, 100, 101, 110, 111)

Intervallo di Valori Rappresetnabili:

Oltre a contare le combinazioni, è importante sapere **quali valori numerici possiamo rappresentare** con N bit.

Con N bit, i valori rappresentabili vanno da `0` a .

Esempi:

o 1 bit: = 1, valori rappresentabili nell'intervallo `[0,1]`
o 2 bit: = 3, valori rappresentabili nell'intervallo `[0,3]`
o 3 bit: = 7, valori rappresentabili nell'intervallo `[0,7]` .

All'interno del dispositivo, tutto è rappresentato da un insieme di cifre binarie. Più specificamente:

- Qualsiasi Sistema Operativo (ad esempio, Windows, Android, Mac OS, Linux)
- Qualsiasi Programma/Applicazione (ad esempio, Word, Excel, Skype, Firefox)
- Qualsiasi Comando/Interazione (ad esempio, Tastiera, Mouse, Touchpad).

ASCII Code

Il Codice Standard Americano per la Codifica delle Informazioni (ASCII) utilizza tipicamente 7 bit per rappresentare ogni carattere, il che consente di avere 128 codici di caratteri unici.
ASCII Esteso: **ASCII a volte viene esteso a 8 bit, il che permette di aggiungere 128 caratteri in più, portando il totale a 256** .
Questa estensione non è standardizzata allo stesso modo dell'originale ASCII a 7 bit, e ci sono vari set di ASCII esteso, come ISO 8859-1 o Windows-1252, che includono caratteri per lingue specifiche o simboli grafici.

Tipi di Caratteri:
• **Caratteri Alfanumerici**: **Comprende tutte le lettere maiuscole e minuscole dell'alfabeto inglese e i numeri (`0-9`).**
• **Simboli**: ASCII **include un insieme di simboli di punteggiatura comuni e altri simboli vari come @, #, $, ecc.**
• **Caratteri di Controllo**: **Questi sono caratteri non stampabili che controllano il flusso del testo o il suo processo in qualche modo** .
Esempi includono TAB (tabulazione orizzontale), LF (line feed), CR (ritorno a capo) e BEL (campanello/avviso).

Comprendere le Dimensioni dei Dati Digitali

Questa tabella fornisce un confronto tra le dimensioni dei dati utilizzando diversi prefissi, le loro dimensioni decimali, le approssimazioni binarie e gli esempi pratici.

• Un gruppo di 8 bit è chiamato Byte;

Abbr.	Prefix-Byte	Decimal Size	Size in Thousands	Binary Approximation	An Example
K	Kilo-	10^3	$1,000^1$	$1,024 = 2^{10}$	A text file (e.g., 1 KB)
M	Mega-	10^6	$1,000^2$	$1,024^2 = 2^{20}$	A song (e.g., MP3 file, 4 MB)
G	Giga-	10^9	$1,000^3$	$1,024^3 = 2^{30}$	A HD film (e.g., MP4 file, 1.5 GB)
T	Tera-	10^{12}	$1,000^4$	$1,024^4 = 2^{40}$	A large backup drive (e.g., 1 TB of data)
P	Peta-	10^{15}	$1,000^5$	$1,024^5 = 2^{50}$	Storage of large-scale research data (e.g., climate data, 1 PB)
E	Exa-	10^{18}	$1,000^6$	$1,024^6 = 2^{60}$	Entire data usage of a major tech company (e.g., Google, 1 EB)

Tab bit.png

All'interno di ogni dispositivo, le informazioni sono rappresentate da un insieme fisso di byte:
• 16 bit (2 byte)
• 32 bit (4 byte)
• 64 bit (8 byte)

Il numero di byte indica la "potenza" di un dispositivo.
Maggiore è il numero di byte:
• **Maggiore è la capacità del dispositivo di eseguire calcoli complessi**
• Maggiore è **la capacità del dispositivo di gestire grandi quantità di informazioni**
• Maggiore è **la capacità del dispositivo di elaborare istruzioni complesse.**

Le proprietà del Codice Binario

Come ogni sistema numerico, il codice binario ha le sue proprietà intrinseche.
Una proprietà chiave, condivisa con altri sistemi posizionali, è la capacità di convertire numeri da una base numerica a un'altra.
Inoltre, il sistema binario supporta le quattro operazioni di base:

- Addizione;
- Sottrazione;
- Moltiplicazione;

- Divisione.

> **NOTA:**
>
> Un numero rappresentato con due cifre è chiamato numero binario (base due).
> Un numero rappresentato con dieci cifre è chiamato numero decimale (base dieci).

Calcoli aritmetici con il codice binario

Conversione decimale in Binario

Per convertire un numero decimale (base 10) in un numero binario (base 2), segui questi passaggi:

1. Dividi il numero per 2.
2. Annota il resto (0 o 1). Questo sarà il bit meno significativo (LSB).
3. Continua a dividere il quoziente ottenuto nel passaggio precedente per 2, annotando i resti.
4. Ripeti fino a quando il quoziente non è 0.
5. Scrivi i resti in ordine inverso: dal più recente al primo ottenuto.
   Questo rappresenta il numero in binario.

Conversione da Decimale a Binario
Convertiamo **13** in binario:
13 ÷ 2 = 6, resto = 1
6 ÷ 2 = 3, resto = 0
3 ÷ 2 = 1, resto = 1
1 ÷ 2 = 0, resto = 1
Risultato in binario: $\mathbf{1101}_2$.

> **NOTA**
>
> Si parte dal basso verso l'alto per leggere il numero binario, difatti il risultato è 1101

Conversione Binario in decimale

Per convertire un numero binario (base 2) in un numero decimale (base 10), segui questi passaggi:

6. Scrivi il numero binario.
7. Elenca le potenze di 2 da destra a sinistra, iniziando con sotto il bit più a destra.
8. Moltiplica ogni bit per la corrispondente potenza di 2.
9. Somma i risultati per ottenere il numero decimale.

Conversione da Binario a Decimale
Convertiamo **1101_2** in decimale:
1101 = 1 · 2^3 + 1 · 2^2 + 0 · 2^1 + 1 · 2^0
= 8 + 4 + 0 + 1 = 13
Risultato in decimale: **13_{10}**.

L'addizione binaria

Prendi i due numeri binari che vuoi sommare.

> **NOTA: Assicurati che entrambi i numeri abbiano la stessa lunghezza.**
>
> Se non lo sono, aggiungi zeri iniziali al numero più corto fino a che non abbiano la stessa lunghezza.

1. Inizia a sommare i bit dalla parte destra (bit meno significativo) dei due numeri binari, seguendo queste regole:

Case	A + B	Sum	Carry
1	0 + 0	0	0
2	0 + 1	1	0
3	1 + 0	1	0
4	1 + 1	0	1

Bin add.png

2. Continua sommando i bit successivi e considerando eventuali riporto dalle colonne precedenti. Se dopo aver sommato l'ultimo bit rimane un riporto,

scrivilo nella colonna più a sinistra del risultato.
Esempio dell'addizione binaria:

```
Binary Addition/2
Example 1 (1011 + 1101):
1011
+ 1101
-----------
11000 (Result: carry applied)
```

```
Step-by-step addition with carry:
1 + 1 = 0 (carry 1)
1 + 0 + 1 (carry) = 0 (carry 1)
0 + 1 + 1 (carry) = 0 (carry 1)
1 + 1 + 1 (carry) = 1 (carry 1 to a new
column)
```

```
Example 2 (1011 + 101):
1011
+ 0101
-----------
10000 (Result: carry applied)
```

```
Step-by-step addition with carry:
1 + 1 = 0 (carry 1)
1 + 0 + 1 (carry) = 0 (carry 1)
0 + 1 + 1 (carry) = 0 (carry 1)
1 + 0 + 1 (carry) = 0 (carry 1 to a new
column
```

Sottrazione Binaria

Prendi i due numeri binari che vuoi sottrarre.

> **NOTA: Assicurati che entrambi i numeri abbiano la stessa lunghezza.**
>
> Se non lo sono, aggiungi zeri iniziali al numero più corto fino a che non abbiano la stessa lunghezza.

12. Inizia a sottrarre i bit dalla parte destra (bit meno significativo) dei due numeri binari, seguendo queste regole:

Case	A - B	Difference	Borrow
1	0 - 0	0	0
2	1 - 0	1	0
3	1 - 1	0	0
4	0 - 1	1	1

Bin sub.png

13. Continua sottraendo i bit successivi e considerando eventuali prestiti dalle colonne precedenti. Se dopo aver sottratto l'ultimo bit rimane un prestito, scrivilo nella colonna più a sinistra del risultato.

```
Example 1 (1110 - 1001):
1110
- 1001
---------
0101 (Result: borrow applied)
```

```

Step-by-step subtraction with borrow:
0 - 1 = 1 (borrow 1)
1 - 0 - 1 (borrow) = 0
1 - 0 = 1
1 - 1 = 0
```

```
Example 2 (1011 - 101):
1011
- 0101
-----------
0110 (Result: borrow applied)
```

```
Step-by-step subtraction with borrow:
1 - 1 = 0
1 - 0 = 1
0 - 1 = 1 (borrow 1)
1 - 0 - 1 (borrow) = 0
```

L'algebra Booleana

L'Algebra Booleana è un ramo dell'algebra che si occupa di valori veri o falsi, tipicamente denotati come `1` e `0`, rispettivamente.
È fondamentale nel campo dell'informatica e dell'elettronica digitale perché viene utilizzata per progettare e analizzare il comportamento di circuiti digitali e porte logiche.

> ⓘ **Concetti chiave:**
>
> 1. Variabili binarie:
> Rappresentate come 1 (vero) e 0 (falso).
> 2. Operazioni logiche:
>
> - AND (*):
> **Restituisce vero se entrambi gli operandi sono veri (1 * 1 = 1).**
> - OR (+):
> **Restituisce vero se almeno uno degli operandi è vero (1 + 0 = 1).**
> - NOT (¬):
> **Restituisce l'inverso dell'operando (¬1 = 0) (è il segno meno).**

Tabelle di verità

Le tabelle di verità servono a rappresentare il comportamento degli operatori logici (AND, OR, NOT, ecc.) applicati a variabili booleane, cioè che possono assumere solo due valori:

- `0` → **falso**
- `1` → **vero**

Operatore **AND** (E logico)

La condizione è vera (1) solo se entrambi gli operandi sono veri.

A	B	A AND B
0	0	0
0	1	0
1	0	0

A	B	A AND B
1	1	1

Esempio Pratico:

"Devo uscire se **ho i soldi** AND **ho tempo libero**". Se manca anche solo una condizione, non posso uscire.

Operatore **OR** (O logico)

La condizione è vera (1) se almeno uno dei due operandi è vero.

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

Esempio Pratico

"Prendo un gelato se c'è gusto cioccolato OR gusto vaniglia". Basta che sia disponibile uno dei due.

Operatore **NOT** (Negazione)

Inverte il valore logico: da vero a falso, da falso a vero.
In sostanza l'operatore **NOT** ha il compito di **invertire** il valore logico della variabile a cui si applica:

- **se la variabile vale `1` (vero), dopo la negazione diventa `0` (falso);**
- **se la variabile vale `0` (falso), dopo la negazione diventa `1` (vero).**

A	NOT A
0	1

A	NOT A
1	0

Possiamo vedere il **NOT** come un **interruttore logico:**

- **Se l'affermazione è vera (1), il NOT la trasforma in falsa (0).**
- **Se l'affermazione è falsa (0), il NOT la trasforma in vera (1).**

In pratica capovolge sempre il significato logico di ciò a cui si applica.

Esempio pratico

se "oggi NON piove", significa che il tempo è bello.

Come funziona nella logica booleana

- Frase: "Oggi **piove**" → **vero (1)**.
  - Applicando il NOT: "Oggi **NON piove**" → **falso (0)**.
- Frase: "Il computer è acceso" → **vero (1)**.
  - Applicando il NOT: "Il computer è **spento**" → **falso (0)**.
- Frase: "La porta è chiusa" → **falso (0)**.
  - Applicando il NOT: "La porta **non è chiusa**" (cioè è aperta) → **vero (1)**.

Esempio informatico

Immaginiamo di avere una lampadina collegata a un interruttore logico:

- Se la variabile `LUCE = 1` (accesa), allora `NOT LUCE = 0` (spenta).
- Se la variabile `LUCE = 0` (spenta), allora `NOT LUCE = 1` (accesa).

Il NOT in questo caso rappresenta un **interruttore che capovolge lo stato**.

Proprietà fondamentali

1. **Commutative:**
   scambiare l'ordine degli operandi non cambia il risultato.
   Vale per **AND** e **OR**, **non** ha senso per **NOT** (che è unario)
   In altre parole l'ordine degli operandi non cambia il risultato.
   - `A OR B = B OR A`
   - `A AND B = B AND A`

Esempio intuitivo

- `A OR B`: "Piove **o** c'è vento" è equivalente a "C'è vento **o** piove".
- `A AND B`: "Ho tempo **e** ho soldi" ≡ "Ho soldi **e** ho tempo".

⚠ Nota operativa

La commutatività è utile per **riordinare** i termini prima di una semplificazione (es. raggruppare variabili uguali).

2. Distributiva:
   - **AND e OR possono "distribuirsi" l'uno sull'altro, come accade nell'algebra aritmetica.**

Intuizione con insiemi

- `OR` ↔ **unione**; `AND` ↔ **intersezione**.
- `A ∪ (B ∩ C) = (A ∪ B) ∩ (A ∪ C)` e
  `A ∩ (B ∪ C) = (A ∩ B) ∪ (A ∩ C)`.

3. De Morgan's Law:
   - **la negazione "capovolge" AND ↔ OR e nega i singoli operandi.**
   - `NOT (A AND B) = (NOT A) OR (NOT B)`
   - `NOT (A OR B) = (NOT A) AND (NOT B)`

Lettura in lingua naturale

- “**Non** (A **e** B)” ≡ “(**Non** A) **o** (**Non** B)”.
  Es.: “Non (piove **e** c’è vento)” ≡ “Non piove **o** non c’è vento”.
- “**Non** (A **o** B)” ≡ “(**Non** A) **e** (**Non** B)”.
  Es.: “Non (luci accese **o** porta aperta)” ≡ “Luci spente **e** porta chiusa”.

⚠ Le regole di precedenza

Spesso accadde di avere più operatori logici nella stessa espressione, l’ordine con cui vengono eseguiti è:

1. **NOT** → ha la precedenza più alta.
2. **AND** → viene valutato dopo.
3. **OR** → ha la precedenza più bassa.

Per modificare la precedenza si usano le parentesi, che hanno sempre priorità su tutto.

Esempi mirati

- `NOT A OR B` → prima `NOT A`, poi `(... ) OR B`.
- `A AND B OR C` → prima `A AND B`, poi `(... ) OR C`.
- `NOT (A OR B)` → per le parentesi si valuta `A OR B`, **poi** la negazione.

⚠ Errori comuni

- Dimenticare che **NOT** ha precedenza massima: `NOT A AND B` **non** è `(NOT (A AND B))`, bensì `(NOT A) AND B`.
- Omettere parentesi in espressioni lunghe: meglio abbondare per evitare ambiguità.